SPRINT 5 COMPLETION REPORT

SHADOW OS v2 — Shadow LAB Foundation

' COMPLETE — 27/27 TESTS PASS

Date: 2026-07-11 · Sprint 5 · FOREX ENGINE PRO

Executive Summary

Sprint 5 builds the **Shadow LAB Foundation**
: a research-only measurement layer
that reconciles the append-only events stream into structured, fully-provenanced
research tables and computes trade **expectancy**
over them. It answers one
question — *"what is the system actually learning?"* — without ever touching a
single live-trading decision.

The layer is **contract-first, additive, append-first, idempotent, and
reversible**. It is gated behind a new flag SHADOW_LAB_RESEARCH that defaults
off, in which state the reconciler never starts and the system behaves exactly
as it did before this sprint. Every research row carries a full provenance triple
(run_id, build_id, config_hash) plus a dedupe_key, so every measurement is
reproducible from the exact code and configuration that produced it, and every
write is idempotent.

The production entrypoint index.js — which holds the live Engine A/B/C/D
decisions — was **not modified**
(its git diff is empty). No DROP, DELETE, or
TRUNCATE exists anywhere in the migration or manager code.

**Verdict: ' COMPLETE — 23 new Sprint 5 tests + 4 Sprint 4.1 regression tests all
passing (27/27); index.js provably untouched; flag-off proven to be a complete
no-op.**

Objective & Binding Constraints

Deliver a measurement layer over the existing event-sourced Shadow LAB with
zero risk to live trading. The Sprint operated under a binding constitution:

- **Zero changes** to entries/exits/SL/TP/BE/trailing/risk/filters or the Engine A/B/C/D decisions. index.js stays FROZEN.
- **No DROP / DELETE / TRUNCATE** anywhere. All DDL uses IF NOT EXISTS / ADD COLUMN IF NOT EXISTS. (Test teardown deletes only its own namespaced seed rows, mirroring autoMigrate.test.js.)
- **Flag-gated:** SHADOW_LAB_RESEARCH off = a complete no-op.
- **Full provenance:** every research row carries run_id + build_id + config_hash + dedupe_key; config_hash is deterministic.
- **events.data handled as BOTH TEXT and JSONB** (production is TEXT).
- **Additive / append-first / idempotent / reversible / contract-first.**
- **confidence_level auto-computed and tested.**
- Each phase green before the next.

What Changed

The work was delivered in seven phases (F1–F7), each gated on green tests.

1. Schema — telemetry/migrations/005_shadowlab_foundation.sql (F1)

Four new research tables, all CREATE TABLE IF NOT EXISTS, each with the provenance triple, a dedupe_key, and supporting indexes:

- **shadow_signals** — one row per observed signal (from trade_open); full raw signal preserved in a features JSONB column plus promoted scalar columns for fast filtering.
- **shadow_engine_evals** — one row per engine evaluation (from lab_shadow_a/b/c/d); would_trade is a nullable BOOLEAN so engine abstention is preserved as NULL.
- **shadow_outcomes** — one row per resolved trade (from trade_close).
- **shadow_expectancy_snapshots** — append-first expectancy time series; one row per distinct (config_hash, scope, resolved_trades).

Registered in telemetry/migrations/autoMigrate.js so it auto-applies idempotently on startup (pg simple-protocol, no psql dependency), and added to the autoMigrate.test.js expected-tables set.

2. Provenance module — telemetry/managers/shadowLabProvenance.js (F2)

- **configHash(env)** — SHA-256 over a canonical, sorted-key JSON of the decision-relevant configuration surface plus the system version. Deterministic and independent of environment-key insertion order.
- **resolveBuildId(env)** — reproducible <version>+<git-sha|unknown> build id.
- **runId** — a per-process UUID.
- **confidenceLevel(n)** — LOW (<30), MEDIUM (30–100), HIGH (>100).
- **createProvenance({...})** — returns { runId, buildId, configHash, stamp() } where stamp(row) decorates a row with the provenance triple.

3. Reconciler — telemetry/managers/ShadowLabManager.js (F3)

A cursor-based, idempotent, best-effort reconciler over the events stream:

- Projects trade_open !' shadow_signals, `lab_shadow_a/b/c/d !' shadow_engine_evals, trade_close !' shadow_outcomes`.
- **Idempotent** via ON CONFLICT (dedupe_key) DO NOTHING; re-reconciling the same events inserts nothing new.
- **Best-effort**: every projection is wrapped in try/catch; one bad row can never stall the cursor or throw into a caller.
- **Handles events.data as both TEXT and JSONB** via a single parseData

helper.

- **Resumable cursor** persisted as an append-only events row of type shadowlab_research_cursor; recoverCursor() resumes after a restart.
- Lifecycle: reconcileOnce / reconcileAll / recoverCursor / start / stop / getStats. The polling timer is unref'd so it never keeps the process alive.

4. Expectancy — compute, snapshot & read APIs (F4)

Added to ShadowLabManager:

- **computeExpectancy(scope)** — over resolved outcomes: expectancy_pips = total_profit_pips / resolved_trades; win = profit_pips > 0, loss = < 0, breakeven = 0; profit_factor = gross_profit / gross_loss (null when there are no losses).
- **snapshotExpectancy(scope)** — appends an idempotent time-series point keyed exp:<config_hash>:<scope>:<resolved_trades>; called best-effort from reconcileOnce whenever new trades resolve.
- **getExpectancy / getResearchSummary / getTimeseries** — read APIs for the endpoints.

Design decision (locked): confidence_level is computed from **resolved_trades**, not sample_count. Because the snapshot dedupe key is (config_hash, scope, resolved_trades), tying confidence to resolved_trades keeps the snapshot self-consistent — the same resolved_trades always yields an identical snapshot — and is the statistically honest sample for an expectancy figure. Basing it on sample_count would make two snapshots with the same dedupe key carry different confidence, breaking deterministic dedupe.

Bug fixed during F3/F4: the numeric coercion helper returned 0 for null/undefined/"" (because Number(null) === 0), which would silently turn an abstaining engine's "no data" winrate into a real 0. It now returns null for those, and boolean coercion stays tri-state (true/false/null) so engine abstention is never fabricated into a decision.

5. Wiring — flag-gated (F5)

- Barrel export of ShadowLabManager + provenance helpers from telemetry/managers/index.js.
- telemetry/server.js: a flag SHADOW_LAB_RESEARCH (default **off**) gates the reconciler start inside app.listen. Three **read-only, additive** endpoints — /api/lab/expectancy, /api/lab/research/summary, /api/lab/research/timeseries — are always registered and report researchEnabled so a caller knows whether the tables are being actively

populated. When the flag is off, the reconciler never starts (zero behavior change); the manager instance itself is side-effect-free until start().

6. Verification (F6)

Full Sprint 5 suite plus the Sprint 4.1 autoMigrate regression run green, and a git diff proves the change set is additive-only with index.js untouched.

7. Deliverables (F7)

This report (+ PDF), CHANGELOG.md, replit.md, and docs/architecture/MASTER_ARCHITECTURE.md updates, and an architect review.

Requirements Compliance

#	Requirement	Status
1	Zero changes to live trading / Engine A/B/C/D decisions	' (index.js diff empty)
2	index.js FROZEN	' (untouched)
3	Additive / append-first / idempotent / reversible	' (all DDL IF NOT EXISTS; ON CONFLICT DO NOTHING)
4	Flag SHADOW_LAB_RESEARCH off = no-op	' (reconciler never starts; proven via smoke test)
5	No DROP / DELETE / TRUNCATE	' (none in migration/manager; test deletes only own seed rows)
6	Every research row carries run_id+build_id+config_hash+dedupe_key	' (provenance stamp() on every insert)
7	Deterministic config_hash	' (SHA-256, canonical sorted-key JSON; tested)
8	events.data handled as BOTH TEXT and JSONB	' (parseData; tested both shapes)
9	confidence_level auto-computed + tested	' (from resolved_trades; LOW/MEDIUM/HIGH tested)
10	Each phase green before the next	' (F1!'F7)

Verification

Full suite run against the development PostgreSQL (node --test --test-reporter=spec --test-concurrency=1):

```
'  ensureSchema creates the full v2 schema and is idempotent + data-safe (4 tests)
'  ShadowLabManager reconciles the event stream into research tables (7 tests)
'  ShadowLabManager expectancy aggregates (7 tests)
'  shadowLabProvenance determinism / tiers / build id / stamp (9 tests)
!9 tests 27  !9 pass 27  !9 fail 0
```

Highlights proven by the suite:

- **Idempotency** — re-reconciling the same events inserts zero new rows; re-snapshotting unchanged data is a no-op.
- **Provenance** — every row carries the stamped triple; config_hash is a

64-char hex, deterministic, and insertion-order-independent.

- **Abstention preserved** — an engine wouldTrade: null stores `would_trade IS NULL (not false); a null winrate stores NULL (not 0`).
- **Expectancy math** — a known set [+10, +20, " 5, 0] yields `resolved=4, wins=2, losses=1, breakeven=1, total=25, expectancy=6.25, profit_factor=6`; a no-loss set yields profit_factor = null.
- **Confidence tiers** — 4 !' LOW, 50 !' MEDIUM, 101 !' HIGH.
- **Time series** — a newly resolved trade appends a new snapshot point; ordered oldest!'newest.
- **TEXT + JSONB** — events.data parsed correctly in both shapes.

Flag behaviour was verified by booting the real server.js (live-bot spawn stubbed) and hitting all three endpoints:

- **Flag off (default):** endpoints return 200 with researchEnabled: false; the reconciler never starts.
- **Flag on:** the reconciler starts (recoverCursor resumes), endpoints return researchEnabled: true.

Zero-live-change proven by git diff: index.js shows **0** changed lines; the only touched files are the new research modules/tests, the barrel export, the additive server.js wiring, and documentation.

Design Notes & Rationale

- **Why confidence from resolved_trades, not sample_count:** the snapshot identity (dedupe key) is (config_hash, scope, resolved_trades). Anchoring confidence to resolved_trades keeps snapshots deterministic and makes the confidence reflect the actual sample the expectancy rests on. sample_count (total signals observed) is retained separately for context.
- **Why the reconciler is a separate cursor, not a hook in index.js:** the live entrypoint is FROZEN and the layer must be reversible. A cursor over the append-only events stream means the research layer can be turned on, off, or replayed from scratch with no coupling to — and no risk to — live trading.
- **Why best-effort everywhere:** a measurement-layer failure must never affect trading. Every projection, cursor persist, and snapshot is wrapped in try/catch and logged; failures degrade to no-op.

Residual Notes (non-blocking)

1. **First real population happens on the Railway boot with the flag on.** The dev database already contained some events, so the suite exercised reconciliation

over existing data. Enabling SHADOW_LAB_RESEARCH=on in production begins live population; the operator can confirm via /api/lab/research/summary.

2. **Latent events schema-ownership drift (carried from Sprint 4.1).** telemetry/index.js _initSchema defines events.data TEXT; migration 001 defines events.data JSONB. The research layer is deliberately agnostic (parseData handles both). Harmless today; reconcile before adding any JSONB operators on events.data.
3. **Snapshot cadence is per-reconcile-with-new-outcomes.** The time series grows one point per distinct resolved_trades; there is no wall-clock scheduler. This is intentional for a foundation sprint — a periodic scheduler can be added later without schema change.
4. **Best-effort projection advances past a transiently failed row.** Each projection is try/catch and the cursor moves forward, so a row that fails to insert once is skipped rather than retried. The recovery path is a manual replay from cursor 0, which is safe because every insert is idempotent (ON CONFLICT (dedupe_key) DO NOTHING) — a replay re-projects only the rows that were missed. This is an accepted tradeoff for a research layer that must never stall or throw into the trading path.

Operator Runbook — Enabling the Research Layer

1. Ensure production PostgreSQL is attached (Sprint 4.1). On boot, migration 005 auto-applies idempotently.
2. Set SHADOW_LAB_RESEARCH=on in the Railway service variables and redeploy.
3. On boot the log shows [SERVER] SHADOW_LAB_RESEARCH=on — starting research reconciler (read-only) and [SHADOWLAB-RESEARCH] online — run_id=… build_id=… lastId=….
4. Inspect via:
   - GET /api/lab/research/summary — counts, ALL expectancy, per-engine behaviour.
   - GET /api/lab/expectancy?scope=ALL (or a symbol) — live expectancy.
   - GET /api/lab/research/timeseries?scope=ALL&limit=500 — expectancy over time.
5. To fully revert to pre-Sprint-5 behaviour, unset the flag (or set it to off) and redeploy — the reconciler stops; the research tables remain (append-only).

Files Touched

File	Change
telemetry/migrations/005_shadowlab_foundation.sql	New — 4 research tables + indexes (all IF NOT EXISTS)

telemetry/migrations/autoMigrate.js	Register 005 in the migration set
telemetry/managers/shadowLabProvenance.js	New — provenance (build id, config hash, run id, confidence tiers)
telemetry/managers/ShadowLabManager.js	New — cursor reconciler + expectancy compute/snapshot + read APIs
telemetry/managers/index.js	Barrel-export the Shadow LAB research layer
telemetry/server.js	Flag-gated (SHADOW_LAB_RESEARCH, default off) start + 3 read-only endpoints
telemetry/tests/unit/shadowLabProvenance.test.js	New — provenance determinism/tiers/build-id (9 tests)
telemetry/tests/integration/shadowLabManager.test.js	New — reconciliation/idempotency/abstention (7 tests)
telemetry/tests/integration/shadowLabExpectancy.test.js	New — expectancy math + confidence tiers + snapshots (7 tests)
telemetry/tests/integration/autoMigrate.test.js	Add the 4 new tables to the expected-tables set
docs/reports/SPRINT_5_REPORT.md (+ .pdf)	This report
CHANGELOG.md, replit.md, docs/architecture/MASTER_ARCHITECTURE.md	Documentation updates

Untouched (per constraints): index.js.

FOREX ENGINE PRO · SHADOW OS v2 Migration · Sprint 5 · 2026-07-11